Table 1: Formal specification: ALU32 (37 instructions) — dst ∈ [0,9], src ∈ [0,10].

opcode	dst (r_d)	src (r_s)	ofs	imm	semantics (refer to)	Linux eBPF macro
0x04	[0,9]	0	0	any	ALU32 *add* r_d *imm*	BPF_ALU | BPF_ADD | BPF_K
0x0c	[0,9]	[0,10]	0	0x00	ALU32 *add* r_d r_s	BPF_ALU | BPF_ADD | BPF_X
0x14	[0,9]	0	0	any	ALU32 *sub* r_d *imm*	BPF_ALU | BPF_SUB | BPF_K
0x1c	[0,9]	[0,10]	0	0x00	ALU32 *sub* r_d r_s	BPF_ALU | BPF_SUB | BPF_X
0x24	[0,9]	0	0	any	ALU32 *mul* r_d *imm*	BPF_ALU | BPF_MUL | BPF_K
0x2c	[0,9]	[0,10]	0	0x00	ALU32 *mul* r_d r_s	BPF_ALU | BPF_MUL | BPF_X
0x34	[0,9]	0	0	$\neq 0$	ALU32 *udiv* r_d *imm*	BPF_ALU | BPF_DIV | BPF_K
0x34	[0,9]	0	1	$\neq 0$	ALU32 *sdiv* r_d *imm*	BPF_ALU | BPF_DIV | BPF_K
0x3c	[0,9]	[0,10]	0	0x00	ALU32 *udiv* r_d r_s	BPF_ALU | BPF_DIV | BPF_X
0x3c	[0,9]	[0,10]	1	0x00	ALU32 *sdiv* r_d r_s	BPF_ALU | BPF_DIV | BPF_X
0x44	[0,9]	0	0	any	ALU32 *or* r_d *imm*	BPF_ALU | BPF_OR | BPF_K
0x4c	[0,9]	[0,10]	0	0x00	ALU32 *or* r_d r_s	BPF_ALU | BPF_OR | BPF_X
0x54	[0,9]	0	0	any	ALU32 *and* r_d *imm*	BPF_ALU | BPF_AND | BPF_K
0x5c	[0,9]	[0,10]	0	0x00	ALU32 *and* r_d r_s	BPF_ALU | BPF_AND | BPF_X
0x64	[0,9]	0	0	any	ALU32 *lsh* r_d *imm*	BPF_ALU | BPF_LSH | BPF_K
0x6c	[0,9]	[0,10]	0	0x00	ALU32 *lsh* r_d r_s	BPF_ALU | BPF_LSH | BPF_X
0x74	[0,9]	0	0	any	ALU32 *rsh* r_d *imm*	BPF_ALU | BPF_RSH | BPF_K
0x7c	[0,9]	[0,10]	0	0x00	ALU32 *rsh* r_d r_s	BPF_ALU | BPF_RSH | BPF_X
0x84	[0,9]	0	0	0x00	NEG32 r_d	BPF_ALU | BPF_NEG
0x94	[0,9]	0	0	$\neq 0$	ALU32 *umod* r_d *imm*	BPF_ALU | BPF_MOD | BPF_K
0x94	[0,9]	0	1	$\neq 0$	ALU32 *smod* r_d *imm*	BPF_ALU | BPF_MOD | BPF_K
0x9c	[0,9]	[0,10]	0	0x00	ALU32 *umod* r_d r_s	BPF_ALU | BPF_MOD | BPF_X
0x9c	[0,9]	[0,10]	1	0x00	ALU32 *smod* r_d r_s	BPF_ALU | BPF_MOD | BPF_X
0xa4	[0,9]	0	0	any	ALU32 *xor* r_d *imm*	BPF_ALU | BPF_XOR | BPF_K
0xac	[0,9]	[0,10]	0	0x00	ALU32 *xor* r_d r_s	BPF_ALU | BPF_XOR | BPF_X
0xb4	[0,9]	0	0	any	MOVI32 r_d *imm*	BPF_ALU | BPF_MOV | BPF_K
0xbc	[0,9]	[0,10]	0	0x00	MOV *mov*32*sx*32 r_d r_s	BPF_ALU | BPF_MOV | BPF_X
0xbc	[0,9]	[0,10]	8	0x00	MOV *mov*32*sx*8 r_d r_s	BPF_ALU | BPF_MOV | BPF_X
0xbc	[0,9]	[0,10]	16	0x00	MOV *mov*32*sx*16 r_d r_s	BPF_ALU | BPF_MOV | BPF_X
0xc4	[0,9]	0	0	any	ALU32 *arsh* r_d *imm*	BPF_ALU | BPF_ARSH | BPF_K
0xcc	[0,9]	[0,10]	0	0x00	ALU32 *arsh* r_d r_s	BPF_ALU | BPF_ARSH | BPF_X
0xd4	[0,9]	0	0	0x10	END *le*16 r_d	BPF_ALU | BPF_END | BPF_TO_LE | 0x10
0xd4	[0,9]	0	0	0x20	END *le*32 r_d	BPF_ALU | BPF_END | BPF_TO_LE | 0x20
0xd4	[0,9]	0	0	0x40	END *le*64 r_d	BPF_ALU | BPF_END | BPF_TO_LE | 0x40
0xdc	[0,9]	0	0	0x10	END *be*16 r_d	BPF_ALU | BPF_END | BPF_TO_BE | 0x10
0xdc	[0,9]	0	0	0x20	END *be*32 r_d	BPF_ALU | BPF_END | BPF_TO_BE | 0x20
0xdc	[0,9]	0	0	0x40	END *be*64 r_d	BPF_ALU | BPF_END | BPF_TO_BE | 0x40

Table 2: Formal specification: ALU64 (35 instructions) — dst ∈ [0,9], src ∈ [0,10].

opcode	dst (r_d)	src (r_s)	ofs	imm	semantics (refer to)	Linux eBPF macro
0x07	[0,9]	0	0	any	ALU64 *add* r_d *imm*	BPF_ALU64 | BPF_ADD | BPF_K
0x0f	[0,9]	[0,10]	0	0x00	ALU64 *add* r_d r_s	BPF_ALU64 | BPF_ADD | BPF_X
0x17	[0,9]	0	0	any	ALU64 *sub* r_d *imm*	BPF_ALU64 | BPF_SUB | BPF_K
0x1f	[0,9]	[0,10]	0	0x00	ALU64 *sub* r_d r_s	BPF_ALU64 | BPF_SUB | BPF_X
0x27	[0,9]	0	0	any	ALU64 *mul* r_d *imm*	BPF_ALU64 | BPF_MUL | BPF_K
0x2f	[0,9]	[0,10]	0	0x00	ALU64 *mul* r_d r_s	BPF_ALU64 | BPF_MUL | BPF_X
0x37	[0,9]	0	0	$\neq 0$	ALU64 *udiv* r_d *imm*	BPF_ALU64 | BPF_DIV | BPF_K
0x37	[0,9]	0	1	$\neq 0$	ALU64 *sdiv* r_d *imm*	BPF_ALU64 | BPF_DIV | BPF_K
0x3f	[0,9]	[0,10]	0	0x00	ALU64 *udiv* r_d r_s	BPF_ALU64 | BPF_DIV | BPF_X
0x3f	[0,9]	[0,10]	1	0x00	ALU64 *sdiv* r_d r_s	BPF_ALU64 | BPF_DIV | BPF_X
0x47	[0,9]	0	0	any	ALU64 *or* r_d *imm*	BPF_ALU64 | BPF_OR | BPF_K
0x4f	[0,9]	[0,10]	0	0x00	ALU64 *or* r_d r_s	BPF_ALU64 | BPF_OR | BPF_X
0x57	[0,9]	0	0	any	ALU64 *and* r_d *imm*	BPF_ALU64 | BPF_AND | BPF_K
0x5f	[0,9]	[0,10]	0	0x00	ALU64 *and* r_d r_s	BPF_ALU64 | BPF_AND | BPF_X
0x67	[0,9]	0	0	any	ALU64 *lsh* r_d *imm*	BPF_ALU64 | BPF_LSH | BPF_K
0x6f	[0,9]	[0,10]	0	0x00	ALU64 *lsh* r_d r_s	BPF_ALU64 | BPF_LSH | BPF_X
0x77	[0,9]	0	0	any	ALU64 *rsh* r_d *imm*	BPF_ALU64 | BPF_RSH | BPF_K
0x7f	[0,9]	[0,10]	0	0x00	ALU64 *rsh* r_d r_s	BPF_ALU64 | BPF_RSH | BPF_X
0x87	[0,9]	0	0	0x00	NEG64 r_d	BPF_ALU64 | BPF_NEG
0x97	[0,9]	0	0	$\neq 0$	ALU64 *umod* r_d *imm*	BPF_ALU64 | BPF_MOD | BPF_K
0x97	[0,9]	0	1	$\neq 0$	ALU64 *smod* r_d *imm*	BPF_ALU64 | BPF_MOD | BPF_K
0x9f	[0,9]	[0,10]	0	0x00	ALU64 *umod* r_d r_s	BPF_ALU64 | BPF_MOD | BPF_X
0x9f	[0,9]	[0,10]	1	0x00	ALU64 *smod* r_d r_s	BPF_ALU64 | BPF_MOD | BPF_X
0xa7	[0,9]	0	0	any	ALU64 *xor* r_d *imm*	BPF_ALU64 | BPF_XOR | BPF_K
0xaf	[0,9]	[0,10]	0	0x00	ALU64 *xor* r_d r_s	BPF_ALU64 | BPF_XOR | BPF_X
0xb7	[0,9]	0	0	any	MOVI64 r_d *imm*	BPF_ALU64 | BPF_MOV | BPF_K
0xbf	[0,9]	[0,10]	0	0x00	MOV *mov64sx64* r_d r_s	BPF_ALU64 | BPF_MOV | BPF_X
0xbf	[0,9]	[0,10]	8	0x00	MOV *mov64sx8* r_d r_s	BPF_ALU64 | BPF_MOV | BPF_X
0xbf	[0,9]	[0,10]	16	0x00	MOV *mov64sx16* r_d r_s	BPF_ALU64 | BPF_MOV | BPF_X
0xbf	[0,9]	[0,10]	32	0x00	MOV *mov64sx32* r_d r_s	BPF_ALU64 | BPF_MOV | BPF_X
0xc7	[0,9]	0	0	any	ALU64 *arsh* r_d *imm*	BPF_ALU64 | BPF_ARSH | BPF_K
0xcf	[0,9]	[0,10]	0	0x00	ALU64 *arsh* r_d r_s	BPF_ALU64 | BPF_ARSH | BPF_X
0xd7	[0,9]	0	0	0x10	END *sw16* r_d	BPF_ALU64 | BPF_END | 0x10
0xd7	[0,9]	0	0	0x20	END *sw32* r_d	BPF_ALU64 | BPF_END | 0x20
0xd7	[0,9]	0	0	0x40	END *sw64* r_d	BPF_ALU64 | BPF_END | 0x40

Table 3: Formal specification: JMP (46 instructions) — dst ∈ [0,9], src ∈ [0,10].

opcode	dst (r_d)	src (r_s)	ofs	imm	semantics (refer to)	Linux eBPF macro
0x05	-	0	any	0x00	JA ofs	BPF_JMP | BPF_JA
0x06	-	0	0	any	JA32 imm	BPF_JMP32 | BPF_JA
0x15	[0,9]	0	any	any	JMP $eq\ r_d\ imm$	BPF_JMP | BPF_JEQ | BPF_K
0x16	[0,9]	0	any	any	JMP32 $eq\ r_d\ imm$	BPF_JMP32 | BPF_JEQ | BPF_K
0x1d	[0,9]	[0,10]	any	0x00	JMP $eq\ r_d\ r_s$	BPF_JMP | BPF_JEQ | BPF_X
0x1e	[0,9]	[0,10]	any	0x00	JMP32 $eq\ r_d\ r_s$	BPF_JMP32 | BPF_JEQ | BPF_X
0x25	[0,9]	0	any	any	JMP $gt\ r_d\ imm$	BPF_JMP | BPF_JGT | BPF_K
0x26	[0,9]	0	any	any	JMP32 $gt\ r_d\ imm$	BPF_JMP32 | BPF_JGT | BPF_K
0x2d	[0,9]	[0,10]	any	0x00	JMP $gt\ r_d\ r_s$	BPF_JMP | BPF_JGT | BPF_X
0x2e	[0,9]	[0,10]	any	0x00	JMP32 $gt\ r_d\ r_s$	BPF_JMP32 | BPF_JGT | BPF_X
0x35	[0,9]	0	any	any	JMP $ge\ r_d\ imm$	BPF_JMP | BPF_JGE | BPF_K
0x36	[0,9]	0	any	any	JMP32 $ge\ r_d\ imm$	BPF_JMP32 | BPF_JGE | BPF_K
0x3d	[0,9]	[0,10]	any	0x00	JMP $ge\ r_d\ r_s$	BPF_JMP | BPF_JGE | BPF_X
0x3e	[0,9]	[0,10]	any	0x00	JMP32 $ge\ r_d\ r_s$	BPF_JMP32 | BPF_JGE | BPF_X
0x45	[0,9]	0	any	any	JMP $set\ r_d\ imm$	BPF_JMP | BPF_JSET | BPF_K
0x46	[0,9]	0	any	any	JMP32 $set\ r_d\ imm$	BPF_JMP32 | BPF_JSET | BPF_K
0x4d	[0,9]	[0,10]	any	0x00	JMP $set\ r_d\ r_s$	BPF_JMP | BPF_JSET | BPF_X
0x4e	[0,9]	[0,10]	any	0x00	JMP32 $set\ r_d\ r_s$	BPF_JMP32 | BPF_JSET | BPF_X
0x55	[0,9]	0	any	any	JMP $neq\ r_d\ imm$	BPF_JMP | BPF_JNE | BPF_K
0x56	[0,9]	0	any	any	JMP32 $neq\ r_d\ imm$	BPF_JMP32 | BPF_JNE | BPF_K
0x5d	[0,9]	[0,10]	any	0x00	JMP $neq\ r_d\ r_s$	BPF_JMP | BPF_JNE | BPF_X
0x5e	[0,9]	[0,10]	any	0x00	JMP32 $neq\ r_d\ r_s$	BPF_JMP32 | BPF_JNE | BPF_X
0x65	[0,9]	0	any	any	JMP $sgt\ r_d\ imm$	BPF_JMP | BPF_JSGT | BPF_K
0x66	[0,9]	0	any	any	JMP32 $sgt\ r_d\ imm$	BPF_JMP32 | BPF_JSGT | BPF_K
0x6d	[0,9]	[0,10]	any	0x00	JMP $sgt\ r_d\ r_s$	BPF_JMP | BPF_JSGT | BPF_X
0x6e	[0,9]	[0,10]	any	0x00	JMP32 $sgt\ r_d\ r_s$	BPF_JMP32 | BPF_JSGT | BPF_X
0x75	[0,9]	0	any	any	JMP $sge\ r_d\ imm$	BPF_JMP | BPF_JSGE | BPF_K
0x76	[0,9]	0	any	any	JMP32 $sge\ r_d\ imm$	BPF_JMP32 | BPF_JSGE | BPF_K
0x7d	[0,9]	[0,10]	any	0x00	JMP $sge\ r_d\ r_s$	BPF_JMP | BPF_JSGE | BPF_X
0x7e	[0,9]	[0,10]	any	0x00	JMP32 $sge\ r_d\ r_s$	BPF_JMP32 | BPF_JSGE | BPF_X
0xa5	[0,9]	0	any	any	JMP $lt\ r_d\ imm$	BPF_JMP | BPF_JLT | BPF_K
0xa6	[0,9]	0	any	any	JMP32 $lt\ r_d\ imm$	BPF_JMP32 | BPF_JLT | BPF_K
0xad	[0,9]	[0,10]	any	0x00	JMP $lt\ r_d\ r_s$	BPF_JMP | BPF_JLT | BPF_X
0xae	[0,9]	[0,10]	any	0x00	JMP32 $lt\ r_d\ r_s$	BPF_JMP32 | BPF_JLT | BPF_X
0xb5	[0,9]	0	any	any	JMP $le\ r_d\ imm$	BPF_JMP | BPF_JLE | BPF_K
0xb6	[0,9]	0	any	any	JMP32 $le\ r_d\ imm$	BPF_JMP32 | BPF_JLE | BPF_K
0xbd	[0,9]	[0,10]	any	0x00	JMP $le\ r_d\ r_s$	BPF_JMP | BPF_JLE | BPF_X
0xbe	[0,9]	[0,10]	any	0x00	JMP32 $le\ r_d\ r_s$	BPF_JMP32 | BPF_JLE | BPF_X
0xc5	[0,9]	0	any	any	JMP $slt\ r_d\ imm$	BPF_JMP | BPF_JSLT | BPF_K
0xc6	[0,9]	0	any	any	JMP32 $slt\ r_d\ imm$	BPF_JMP32 | BPF_JSLT | BPF_K
0xcd	[0,9]	[0,10]	any	0x00	JMP $slt\ r_d\ r_s$	BPF_JMP | BPF_JSLT | BPF_X
0xce	[0,9]	[0,10]	any	0x00	JMP32 $slt\ r_d\ r_s$	BPF_JMP32 | BPF_JSLT | BPF_X
0xd5	[0,9]	0	any	any	JMP $sle\ r_d\ imm$	BPF_JMP | BPF_JSLE | BPF_K
0xd6	[0,9]	0	any	any	JMP32 $sle\ r_d\ imm$	BPF_JMP32 | BPF_JSLE | BPF_K
0xdd	[0,9]	[0,10]	any	0x00	JMP $sle\ r_d\ r_s$	BPF_JMP | BPF_JSLE | BPF_X
0xde	[0,9]	[0,10]	any	0x00	JMP32 $sle\ r_d\ r_s$	BPF_JMP32 | BPF_JSLE | BPF_X

Table 4: Formal specification: CALL (3 instructions) — dst/src unused.

opcode	dst (r_d)	src (r_s)	ofs	imm	semantics (refer to)	Linux eBPF macro
0x85	0	0	0	any	`CallInternal` imm	`BPF_JMP | BPF_CALL`
0x85	0	0x1	0	any	`CallID` imm	`BPF_JMP | BPF_CALL`
0x85	0	0x2	0	any	`CallBTF` imm	`BPF_JMP | BPF_CALL`

Table 5: Formal specification: TailCall + EXIT + ST_NoSpec (3 instructions) — dst/src unused.

opcode	dst (r_d)	src (r_s)	ofs	imm	semantics (refer to)	Linux eBPF macro
0xf5	0	0x3	0	any	`TailCall`	`BPF_JMP | BPF_TAIL_CALL`
0x95	0	0	0	0x00	`EXIT`	`BPF_JMP | BPF_EXIT`
0xc2	0	0	0	0x00	`ST_NoSpec`	`BPF_ST | BPF_NOSPEC`

Table 6: Formal specification: ST (8 instructions) — dst ∈ [0,10] (including r10 stack pointer), src ∈ [0,10] or K.

opcode	dst (r_d)	src (r_s)	ofs	imm	semantics (refer to)	Linux eBPF macro
0x62	[0,10]	0	any	any	`ST` $Mint32\ r_d\ imm\ ofs$	`BPF_ST | BPF_MEM | BPF_W`
0x63	[0,10]	[0,10]	any	0x00	`ST` $Mint32\ r_d\ r_s\ ofs$	`BPF_STX | BPF_MEM | BPF_W`
0x6a	[0,10]	0	any	any	`ST` $Mint16\ r_d\ imm\ ofs$	`BPF_ST | BPF_MEM | BPF_H`
0x6b	[0,10]	[0,10]	any	0x00	`ST` $Mint16\ r_d\ r_s\ ofs$	`BPF_STX | BPF_MEM | BPF_H`
0x72	[0,10]	0	any	any	`ST` $Mint8\ r_d\ imm\ ofs$	`BPF_ST | BPF_MEM | BPF_B`
0x73	[0,10]	[0,10]	any	0x00	`ST` $Mint8\ r_d\ r_s\ ofs$	`BPF_STX | BPF_MEM | BPF_B`
0x7a	[0,10]	0	any	any	`ST` $Mint64\ r_d\ imm\ ofs$	`BPF_ST | BPF_MEM | BPF_DW`
0x7b	[0,10]	[0,10]	any	0x00	`ST` $Mint64\ r_d\ r_s\ ofs$	`BPF_STX | BPF_MEM | BPF_DW`

Table 7: Formal specification: LD / LDSX (7 instructions) — dst ∈ [0,9], src ∈ [0,10].

opcode	dst (r_d)	src (r_s)	ofs	imm	semantics (refer to)	Linux eBPF macro
0x61	[0,9]	[0,10]	any	0x00	`LD` $Mint32\ r_d\ r_s$	`BPF_LDX | BPF_MEM | BPF_W`
0x69	[0,9]	[0,10]	any	0x00	`LD` $Mint16\ r_d\ r_s$	`BPF_LDX | BPF_MEM | BPF_H`
0x71	[0,9]	[0,10]	any	0x00	`LD` $Mint8\ r_d\ r_s$	`BPF_LDX | BPF_MEM | BPF_B`
0x79	[0,9]	[0,10]	any	0x00	`LD` $Mint64\ r_d\ r_s$	`BPF_LDX | BPF_MEM | BPF_DW`
0x91	[0,9]	[0,10]	any	0x00	`LDSX` $Mint8\ r_d\ r_s$	`BPF_LDX | BPF_MEMSX | BPF_B`
0x89	[0,9]	[0,10]	any	0x00	`LDSX` $Mint16\ r_d\ r_s$	`BPF_LDX | BPF_MEMSX | BPF_H`
0x81	[0,9]	[0,10]	any	0x00	`LDSX` $Mint32\ r_d\ r_s$	`BPF_LDX | BPF_MEMSX | BPF_W`

Table 8: Formal specification: LD_PROBE / LDSX_PROBE (7 instructions) — dst ∈ [0,9], src ∈ [0,10].

opcode	dst (r_d)	src (r_s)	ofs	imm	semantics (refer to)	Linux eBPF macro
0x31	[0,9]	[0,10]	any	0x00	`LD_PROBE` $Mint8\ r_d\ r_s$	`BPF_LDX | BPF_PROBE_MEM | BPF_B`
0x29	[0,9]	[0,10]	any	0x00	`LD_PROBE` $Mint16\ r_d\ r_s$	`BPF_LDX | BPF_PROBE_MEM | BPF_H`
0x21	[0,9]	[0,10]	any	0x00	`LD_PROBE` $Mint32\ r_d\ r_s$	`BPF_LDX | BPF_PROBE_MEM | BPF_W`
0x39	[0,9]	[0,10]	any	0x00	`LD_PROBE` $Mint64\ r_d\ r_s$	`BPF_LDX | BPF_PROBE_MEM | BPF_DW`
0x41	[0,9]	[0,10]	any	0x00	`LDSX_PROBE` $Mint8\ r_d\ r_s$	`BPF_LDX | BPF_PROBE_MEMSX | BPF_B`
0x49	[0,9]	[0,10]	any	0x00	`LDSX_PROBE` $Mint16\ r_d\ r_s$	`BPF_LDX | BPF_PROBE_MEMSX | BPF_H`
0x51	[0,9]	[0,10]	any	0x00	`LDSX_PROBE` $Mint32\ r_d\ r_s$	`BPF_LDX | BPF_PROBE_MEMSX | BPF_W`

Table 9: Formal specification: LDDW (7 instructions) — dst ∈ [0,9], src ∈ [0,7] (pseudo-type subtype).

opcode	dst (r_d)	src (r_s)	ofs	imm	semantics (refer to)	Linux eBPF macro
0x18	[0,9]	0	0	any	LDDW *const* r_d *imm imm*	BPF_LD | BPF_DW | BPF_IMM
0x18	[0,9]	0x1	0	any	LDDW *map_fd* r_d *imm imm*	BPF_LD | BPF_DW | BPF_FD
0x18	[0,9]	0x2	0	any	LDDW *map_v* r_d *imm imm*	BPF_LD | BPF_DW | BPF_FD
0x18	[0,9]	0x3	0	any	LDDW *btf_id* r_d *imm imm*	BPF_LD | BPF_DW | BPF_IMM
0x18	[0,9]	0x4	0	any	LDDW *fptr* r_d *imm imm*	BPF_LD | BPF_DW | BPF_IMM
0x18	[0,9]	0x5	0	any	LDDW *map_id* r_d *imm imm*	BPF_LD | BPF_DW | BPF_IMM
0x18	[0,9]	0x6	0	any	LDDW *map_idv* r_d *imm imm*	BPF_LD | BPF_DW | BPF_IMM